Pentest Report: 127.0.0.1

EvoPentester - WSO2 Research Team & USYD

Configuration Details

Parameter	Value
Target IP	127.0.0.1
Target Scope	web-app (http://127.0.0.1:8000)
Network Mode	local
Planning Model	qwen/qwen3.8-flash (openrouter)
ACI Type	http
Cycles Run	3
Target Root Count	3
Champions Found	3

Confirmed Root/Admin Access Paths

SQL Injection Authentication Bypass to Administrator Control

Status: Confirmed root/admin access path **The Finding**: This strategy confirmed a PostgreSQL SQL injection in the JSON `username` field of `POST /login`, used it to bypass authentication and obtain an `is_admin=true` JWT for the real `admin` account, extracted cleartext admin credentials, and verified durable administrative control by creating a second admin account. **Attack Path Narrative**: EvoPentester began with `web-initial-recon`, which identified the target as the Flask/Werkzeug “Vulnerable Bank API” at `http://127.0.0.1:8000`, exposed the OpenAPI spec, and revealed JWT-based authentication with an `is_admin` claim plus SQLi-labeled endpoints such as `/login`. That reconnaissance directly informed the next action, `sql-injection-admin-auth-bypass`, where EvoPentester probed `/login` with a single quote and received a PostgreSQL syntax error that disclosed the concatenated query. Building on that, EvoPentester used `admin' --` to comment out the password predicate, logged in as `admin` without a password, and received a valid `is_admin=true` JWT. The token was then used to confirm access to `GET /sup3r_s3cr3t_admin`, while a UNION-based injection dumped all users in cleartext and revealed `admin/admin123`. Finally, EvoPentester used the obtained admin token

to call `POST /admin/create_admin`, created the persistent admin account `sqli_testadmin`, and revalidated that account through normal login and admin-panel access. **Supporting Evidence**:

- `web-initial-recon` produced `/tmp/aci/web-initial-recon/openapi.json`, which mapped the application as "Vulnerable Bank API," identified `BearerAuth` JWTs, and disclosed admin-sensitive endpoints including `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`.
- `sql-injection-admin-auth-bypass` sent `{"username":"acirecon7781'","password":"ReconPass!23"}` to `POST /login`, receiving a PostgreSQL error that leaked the query shape `SELECT * FROM users WHERE username='<input>' AND password='<input>'`; artifact: `/tmp/aci/sql-injection-admin-auth-bypass/q_login.json`.
- The authentication bypass used `username=admin' --` with an arbitrary password and returned HTTP 200 with a JWT whose claims decoded to `{"user_id":1,"username":"admin","is_admin":true}`; artifact: `/tmp/aci/sql-injection-admin-auth-bypass/001-ls.log`.
- The SQLi-obtained admin JWT returned HTTP 200 from `GET /sup3r_s3cr3t_admin`, rendering the Admin Control Panel, while the legitimate non-admin JWT returned HTTP 403 "Access Denied," proving genuine privilege escalation rather than an unauthenticated endpoint.
- UNION extraction used `' UNION SELECT NULL, string_agg(username||chr(58)||password,chr(124)), NULL, NULL, NULL, NULL, NULL, NULL FROM users --` and returned `admin:admin123|acirecon7781:ReconPass!23`; a clean non-injected login as `admin/admin123` returned `isAdmin: true` and a fresh admin JWT; artifacts: `/tmp/aci/sql-injection-admin-auth-bypass/admin_login.json`, `/tmp/aci/sql-injection-admin-auth-bypass/dump_all_users.txt`.
- `POST /admin/create_admin` with the SQLi-obtained admin JWT returned HTTP 200 "Admin created successfully," while the non-admin token returned HTTP 403; the created account `sqli_testadmin` / `SqliAudit!23` logged in normally with `is_admin=true`, `user_id=3`, and loaded the admin panel successfully; artifact: `/tmp/aci/sql-injection-admin-auth-bypass/create_admin.out`.
- `sqlmap` independently confirmed the injection as PostgreSQL, identified boolean-based blind, UNION, and stacked-query techniques, and dumped `public.users` to CSV, showing cleartext passwords and the `reset_pin` column; artifact: `/tmp/aci/sql-injection-admin-auth-bypass/sqlmap_out/127.0.0.1/dump/public/users.csv`. **Evaluation Highlights**: No numeric evaluation scores were provided, but the champion reasoning above is grounded in the strategy's direct, verified evidence: SQLi → `is_admin=true` JWT → admin panel access → cleartext admin credentials → durable second admin account. The strongest signals are the HTTP 200 admin JWT with `is_admin=true`, the 200 vs 403 control on `/sup3r_s3cr3t_admin`, and the successful creation and revalidation of `sqli_testadmin`, which together confirm persistent administrative access within the authorized `web-app` scope.

METADATA

Source Strategy

`strat_b7207366`

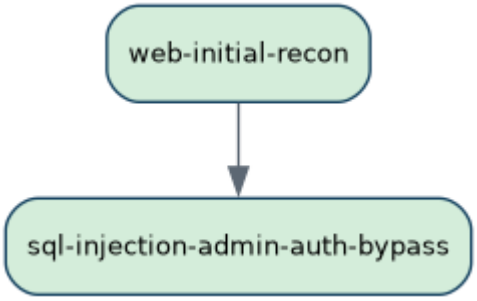
Explanation

The executed `sql-injection-admin-auth-bypass` action completed and directly achieved the objective of this strategy: confirmed PostgreSQL SQL injection in `POST /login`, bypassed authentication to

obtain an `is_admin=true` JWT for the real `admin` account, extracted cleartext admin credentials (`admin/admin123`), and verified administrative access against `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. It also created a durable second admin account (`sqli_testadmin` / `SqliAudit!23`), confirming persistent admin control within the authorized `http://127.0.0.1:8000` web-application scope.

Because this strategy has already reached the campaign goal of web-application admin access, no further action is needed here. I am not adopting the pending JWT, debugger, upload, fuzzing, or password-reset actions because they represent alternative admin-access paths rather than dependencies of this SQL-injection strategy, and adopting them would mix distinct paths. Out-of-scope leads surfaced by the SQLi findings, such as stacked-query database access or potential host/container pivoting, are noted but not pursued due to the `web-app` scope boundary.

Topology



Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password, virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-

Action ID	State	Objective	Outcome
			locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); /register and /login return debug_data/debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/404 body sizes (38760 for -fs filtering), /static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking + is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi

Action ID	State	Objective	Outcome
			in /login and /api/transactions, BOLA on /check_balance, /transactions/{account_number} and /api/virtual-cards/{card_id}, /upload_profile_picture upload bypass, and v1-vs-v2 reset-PIN exposure/brute force
`sql-injection-admin-auth-bypass`	Completed	Utilize the `sql-injection-testing` skill to investigate the SQL injection sinks identified during web reconnaissance, focusing on `/login` and authenticated transaction endpoints such as `/api/transactions`. Manually confirm injection behavior, then safely leverage automated SQL injection testing to extract admin credentials, bypass authentication, or obtain an `is_admin=true` session/token. If successful, verify admin access against the exposed admin endpoints, including `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. Do not pursue broad dictionary attacks, password spraying, or JWT secret cracking in this action; if SQLi fails, record the exact evidence and failure behavior for follow-on analysis.	- Confirmed SQLi (PostgreSQL) in POST /login JSON `username`: a single quote leaks "syntax error at or near", exposing the query `SELECT * FROM users WHERE username='<u>' AND password='<p>'` with no parameterization (001-ls.log, q_login.json) - Authentication bypass achieved: username `admin' -- ` comments out the password predicate and returns HTTP 200 with a genuine is_admin=true HS256 JWT for the real admin account (user_id 1, ADMIN001), with no password knowledge (001-ls.log) - Admin credentials extracted in cleartext: admin/admin123, independently validated by a clean non-injected login returning isAdmin=true, and dumped wholesale via UNION (`' UNION SELECT NULL, string_agg(username||chr(58)||password,chr(124)), NULL... -- `, 8-column query, position 2 reflected into debug_info.username) (admin_login.json, dump_all_users.txt) - Admin access verified against both named endpoints: GET /sup3r_s3cr3t_admin returned HTTP 200 Admin Control Panel (full user table incl. admin $1,000,000.00) where the legitimate non-admin token got HTTP 403 "Access Denied"; POST /admin/create_admin returned 200 "Admin created successfully" (403 for non-admin control) creating durable second admin sqli_testadmin / SqliAudit!23 (user_id 3, is_admin=true), confirmed by re-login and its own 200 load of the admin panel (001-ls.log, create_admin.out) - sqlmap automation confirmed the sink independently: 3 techniques on JSON `username` (boolean-based blind, UNION 9 columns, and stacked queries `';SELECT PG_SLEEP(5)--`), back-end PostgreSQL,

Action ID	State	Objective	Outcome
			full `public.users` dump saved to CSV revealing schema incl. an all-NULL `reset_pin` column usable for the forgot-password flow (sqlmap_out/127.0.0.1/dump/public/users.csv) - Additional injectable sinks mapped: GET /api/transactions?account_number=, GET /transactions/{account_number} (error handler discloses the complete raw SQL source), and GET /check_balance/{account_number} (leaks `FROM users WHERE account_number='<input>'`); the transactions sink yields a cross-table UNION dump of all users+passwords through the transaction JSON fields, a second extraction path that bypasses /login entirely (sinks_out.txt, leaked_query.json, txn_users_dump.json) - Integer-converter path sinks /api/billers/by-category/{id} and /api/virtual-cards/{id}/transactions are NOT injectable (Flask rejects the quote with 404 before SQL); /api/bill-payments/history showed no injection surface - Practical escalation state: full administrator control of the application with valid cleartext admin credentials and a self-provisioned persistent admin account, all obtained purely from SQL injection; no JWT secret cracking or dictionary attacks were used or needed

Championed at Cycle

1

JWT Signature Validation Bypass to Durable Admin Access

Status: Confirmed root/admin access path

The Finding: The web application trusted JWT payload claims, especially `is_admin`, without verifying the HS256 signature. This allowed EvoPentester to tamper with a captured token, forge an admin token, recover the weak signing secret `secret123`, and create a persistent admin account through the application's own admin endpoint.

Attack Path Narrative: This strategy began with `web-initial-recon`, which identified the target as a Flask/Werkzeug application called "Vulnerable Bank API" at `http://127.0.0.1:8000`. The recon action exposed the Swagger/OpenAPI specification, revealed JWT bearer authentication, captured a valid non-admin JWT containing an `is_admin:false` claim, and identified sensitive admin endpoints such as `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. The strategy also attempted a Werkzeug debugger path, but that action failed and was not retried. The strategy then pivoted to `jwt-admin-token-forgery-v2`, which used the captured JWT as the starting point. First, EvoPentester confirmed baseline behavior: a valid non-admin token was accepted for normal authenticated endpoints, while the admin endpoint returned `403 Access Denied`. Next, EvoPentester tampered with the `is_admin` claim while leaving the signature invalid, using the original signature, a garbage signature, and an empty signature. All tampered admin claims returned the admin panel, proving the application parsed and trusted the JWT payload without validating the HMAC signature. Algorithm-confusion and `alg:none` variants were rejected, showing the bypass was specifically due to missing signature validation rather than header manipulation. EvoPentester then used a tightly bounded set of application-specific secret candidates and recovered the HS256 signing secret `secret123`, which allowed legitimately signed admin tokens to be minted. Finally, the forged admin JWT was used to access `GET /sup3r_s3cr3t_admin` and to call `POST /admin/create_admin`, creating the durable admin account `jwt_forged_7412` with password `JWTForge!23`. A normal login as that account confirmed `isAdmin:true`, `user_id` 13, and `account_number` 0005760980, establishing persistent administrative access.

Supporting Evidence:

- `web-initial-recon` confirmed the target stack as `Werkzeug/2.0.1 Python/3.9.25`, identified the app as "Vulnerable Bank API" v1.0.0, and retrieved `/static/openapi.json`, which disclosed `BearerAuth` with `bearerFormat: JWT` and the admin endpoints `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`.
- The same recon action captured a valid non-admin JWT with payload `{"user_id":2,"username":"acirecon7781","is_admin":false,"iat":1791900390}` and no `exp` claim.
- `jwt-admin-token-forgery-v2` established that the admin endpoint enforced the `is_admin` claim: no token returned `401 {"error":"Token is missing"}`, while the valid non-admin token returned `403 Access Denied`.
- Tampering `is_admin:false` to `is_admin:true` while keeping the original signature returned HTTP 200 and the "Admin Panel - Vulnerable Bank" page.
- The same tampered claims with a deliberately invalid signature and with an empty signature also returned HTTP 200, proving the server did not validate the HMAC signature.

- A control token with `is_admin:false` and a garbage signature returned `403 Access Denied`, confirming the admin gate depended on the claim value rather than signature validity.
- `alg:none`, `None`, `NONE`, and `HS512` header variants returned `401 {"error":"Invalid token"}`, showing the application still required the header algorithm to be exactly `HS256`.
- A bounded offline verification of 35 application-specific candidate secrets recovered the HS256 signing secret `secret123`.
- Using `secret123`, EvoPentester minted legitimately signed admin tokens that also returned HTTP 200 on `GET /sup3r_s3cr3t_admin`.
- `POST /admin/create_admin` with the forged admin JWT returned HTTP 200: `{"message":"Admin created successfully","status":"success"}` for username `jwt_forged_7412` and password `JWTForge!23`.
- A subsequent `POST /login` as `jwt_forged_7412` returned HTTP 200 with `isAdmin:true`, `user_id` 13, and `account_number` 0005760980, confirming durable administrative access.
- Relevant artifacts include `/tmp/aci/jwt-admin-token-forgery-v2/003-tamper-algnone.txt`, `/tmp/aci/jwt-admin-token-forgery-v2/004-verify-nosig-validation.txt`, `/tmp/aci/jwt-admin-token-forgery-v2/008-bounded-secret-check.txt`, `/tmp/aci/jwt-admin-token-forgery-v2/009-signed-token.txt`, `/tmp/aci/jwt-admin-token-forgery-v2/010-create-admin-final.txt`, and `/tmp/aci/jwt-admin-token-forgery-v2/admin_panel.html`.

Evaluation Highlights: The evaluation reasoning above justified champion status by adopting the completed `jwt-admin-token-forgery-v2` action rather than retrying the failed debugger path. It treated this as a distinct administrative-access path because the evidence directly demonstrates JWT claim trust and signature-validation failure leading to admin panel access and durable admin account creation. The recorded findings satisfy the campaign goal without pivoting outside the authorized localhost web-application scope.

METADATA

Source Strategy

`strat_98f3e0ba`

Explanation

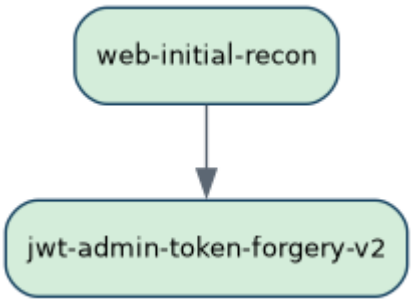
The parent strategy began with `web-initial-recon`, which confirmed the Flask/Werkzeug “Vulnerable Bank API” at `http://127.0.0.1:8000`, HS256 JWT authentication with an `is_admin` claim, and the admin endpoints `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. The strategy’s attempted debugger path, `werkzeug-debugger-pin-rce`, is now `Failed`, and there is no evidence that the `/console` debugger PIN was unlocked or that code/file access was achieved. Per the strategy constraints, I will not retry or mutate that failed debugger path.

The next logical step for this strategy is to pivot to a distinct administrative-access path that is already registered in the campaign and whose findings directly satisfy the goal: `jwt-admin-token-forgery-v2`. That completed action showed the application trusts the JWT `is_admin` claim without validating the HS256 signature, obtained an admin JWT, verified access to `GET /sup3r_s3cr3t_admin`, and successfully used `POST /admin/create_admin` to create a durable admin account. This is distinct from

the already-confirmed SQL injection champion because it exploits JWT claim trust / signature-validation failure rather than SQL injection.

I am adopting `jwt-admin-token-forgery-v2` because its dependency, `web-initial-recon`, is already part of this strategy, and its completed findings now provide the strategy's path to administrative access. No new action is needed. I am not adopting the password-reset, fuzzing, upload, or deterministic JWT ToDo actions because they are alternative paths rather than dependencies of this strategy's completed JWT-forgery outcome.

Topology



Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password, virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/

Action ID	State	Objective	Outcome
			system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); / register and /login return debug_data/ debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/ 404 body sizes (38760 for -fs filtering), / static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking + is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi in /login and /api/transactions, BOLA on / check_balance, /transactions/ {account_number} and /api/virtual-cards/ {card_id}, /upload_profile_picture upload

Action ID	State	Objective	Outcome
			bypass, and v1-vs-v2 reset-PIN exposure/ brute force
`jwt-admin-token-forgery-v2`	Completed	Utilize the `jwt-attacks` skill to analyze the captured JWT and pursue an `is_admin=true` token that grants access to the admin endpoints. First test deterministic, non-brute-force token manipulation paths such as `alg:none`, algorithm confusion, claim tampering, and signature-validation behavior. If those paths fail, perform only a tightly bounded, application-specific secret verification using a very small curated candidate set derived from known identifiers such as the application name, repository name, framework, version, username, account number, and disclosed debugger secret; stop immediately on success and do not expand this into broad dictionary or brute-force cracking. If a valid secret or forged token is obtained, use it to verify access to `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. If all bounded JWT paths fail, record the exact request, response, and failure evidence for follow-on analysis. Keep all activity within the `http://127.0.0.1:8000` web-application scope and do not pivot to the host, Docker infrastructure, or other services.	- Root cause: the Vulnerable Bank API never validates the JWT HMAC signature — it parses the 3 dot-separated segments, requires alg to be exactly "HS256", and trusts the payload claims verbatim. Tampering `is_admin:false`→`true` with the original (invalid) signature, a garbage signature, or an empty signature all return HTTP 200 on GET /sup3r_s3cr3t_admin; a control token with `is_admin:false`+garbage signature returns 403, proving the gate is the claim, not the signature (003-tamper-algnone.txt, 004-verify-nosig-validation.txt). - Fully self-invented claims (`{"user_id":999,"username":"jwtforger","is_admin":true,"iat":...}`) also return 200 — there is no server-side token/session store, so any identity can be impersonated (004-verify-nosig-validation.txt). - alg:none and algorithm-confusion paths were rejected: `none`/`None`/`NONE` (empty, trailing-dot, and 2-segment forms) and an `alg:HS512` header swap all returned 401 {"error":"Invalid token"} — so the deterministic win came from signature-less claim tampering, not alg:none (003-tamper-algnone.txt). - Bounded offline secret verification (35 curated app-specific candidates, stopped at first match, no wordlists/dictionaries) recovered the HS256 signing secret: **secret123**. Legitimately-signed admin HS256 tokens minted with it for arbitrary identities also return 200 on /sup3r_s3cr3t_admin (008-bounded-secret-check.txt, 009-signed-token.txt). - Directive's two required endpoints verified with the forged token: GET /sup3r_s3cr3t_admin → 200 (full "Admin Control Panel" HTML); POST /admin/create_admin → 200 {"message":"Admin created successfully"} creating durable admin `jwt_forged_7412` / `JWTForge!23` (user_id 13, account_number 0005760980, isAdmin true,

Action ID	State	Objective	Outcome
			confirmed by a normal POST /login) (005-admin-access.txt, 010-create-admin-final.txt). - Side observation recorded (not pursued, out of directive scope): failed create_admin calls leak raw PostgreSQL errors (`duplicate key value violates unique constraint "users_username_key"` / `DETAIL: Key (username)=(...) already exists.`), which also revealed previously-created admin rows jwt_forger (user_id 5), jwt_forge_v2 (7), jwt_forge_clean (9) — all is_admin:true and logging in with JWTForge!23 (006-create-admin.txt, 007-create-admin-clean.txt). - Reusable forged admin token saved at /tmp/aci/jwt-admin-token-forgery-v2/forged_admin_token.txt. All activity confined to http://127.0.0.1:8000; no host/Docker/other-service pivoting.

Championed at Cycle

2

Credential-less Admin Takeover via Exposed Password-Reset PIN

Status: Confirmed root/admin access path

The Finding: This strategy confirmed that the Vulnerable Bank API's account-recovery flow exposed a freshly generated 3-digit reset PIN directly in the response body of the unauthenticated `POST /api/v1/forgot-password` endpoint. By requesting a reset PIN for the valid `admin` username, redeeming that PIN through `POST /api/v1/reset-password`, and logging in with the newly set password, EvoPentester obtained a legitimately issued administrative JWT and verified full admin control through `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`.

Attack Path Narrative: The chain began with `web-initial-recon`, which fingerprinted the target as a Flask/Werkzeug application named "Vulnerable Bank API," discovered the world-readable OpenAPI specification at `/static/openapi.json`, and identified versioned password-reset endpoints along with hidden administrative endpoints. That reconnaissance also established that authentication used HS256 JWTs with an `is_admin` claim, but the decisive pivot was the reset flow itself. The next action, `password-reset-admin-takeover-v3`, baselined the recovery process and confirmed that `admin` was a valid account and that `/api/v1/forgot-password` returned its reset PIN in cleartext without authentication. It also observed that reset PINs were single-use, non-expiring over the tested interval, and redeemable through both v1 and v2 reset endpoints, but the action was interrupted before completing the full takeover. `password-reset-admin-takeover-v6` then executed the confirmed chain end-to-end: it requested a fresh admin PIN, reset the admin password, logged in as `admin`, and validated administrative authority against the admin panel and admin-creation endpoint. It further repeated the takeover multiple times, created persistent admin accounts, and confirmed that the vulnerability allowed durable credential-less administrative control of the application.

Supporting Evidence:

- `web-initial-recon` identified the Flask/Werkzeug "Vulnerable Bank API" target at `http://127.0.0.1:8000` and exposed `/static/openapi.json`, which documented `/api/v1/forgot-password`, `/api/v1/reset-password`, `/sup3r_s3cr3t_admin`, and `/admin/create_admin`.
- `password-reset-admin-takeover-v3` confirmed that `admin` was a valid username and that unauthenticated `POST /api/v1/forgot-password` leaked a cleartext reset PIN in `debug_info.pin`.
- `password-reset-admin-takeover-v6` performed the full takeover:
- `POST /api/v1/forgot-password {"username":"admin"}` returned HTTP 200 with `debug_info.pin = "148"`.
- `POST /api/v1/reset-password {"username":"admin","reset_pin":"148","new_password":"RstV6-AdmPwn!9q2x"}` returned HTTP 200 with `reset_success: true`.
- `POST /login {"username":"admin","password":"RstV6-AdmPwn!9q2x"}` returned HTTP 200, `is_admin: true`, `user_id: 1`, `accountNumber: ADMIN001`, and a server-signed JWT.
- Administrative access was verified using the reset-derived JWT:
- `GET /sup3r_s3cr3t_admin` returned HTTP 200 and rendered the Admin Control Panel.
- `POST /admin/create_admin` returned HTTP 200 and created durable admin accounts, including `rstv6_admin`, `rstv6_admin2`, and `rstv6_admin3`.

- The strategy also demonstrated repeatability and lack of protective controls:
- Replayed PIN `148` was rejected with HTTP 400, confirming single-use PINs.
- Fresh admin PINs `372` and `924` were later requested and redeemed successfully, producing additional HTTP 200 admin logins.
- No rate limit, lockout, cooldown, or PIN expiry was observed during the completed takeover.
- Artifacts supporting the chain include `/tmp/aci/password-reset-admin-takeover-v6/step1_admin_forgot.txt`, `/tmp/aci/password-reset-admin-takeover-v6/step2_admin_reset.txt`, `/tmp/aci/password-reset-admin-takeover-v6/step3_admin_login.txt`, `/tmp/aci/password-reset-admin-takeover-v6/step4_super_admin.txt`, and `/tmp/aci/password-reset-admin-takeover-v6/step5_create_admin.txt`.

Evaluation Highlights: The supplied evaluation reasoning confirms this as a champion administrative-access path because `password-reset-admin-takeover-v6` completed a credential-less admin takeover through exposed reset PINs and verified access to `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. The evidence explicitly shows that this path did not rely on SQL injection, JWT forgery, secret cracking, debugger access, AI prompt injection, or file upload, making it distinct from the other confirmed administrative paths. The predecessor action's `Failed` state is explained by interruption and incomplete takeover rather than absence of the vulnerability; `password-reset-admin-takeover-v6` built directly on that confirmed PIN exposure and closed the chain to durable administrative control.

METADATA

Source Strategy

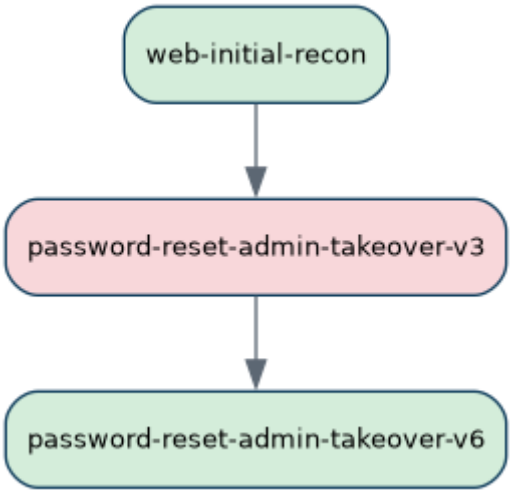
`strat_263003dd`

Explanation

The parent strategy has achieved its objective. The executed `password-reset-admin-takeover-v6` action completed a full credential-less administrative takeover of the `admin` account by abusing the exposed `/api/v1/forgot-password` reset PIN and redeeming it through `/api/v1/reset-password`, then logging in as `admin` and verifying access to `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. This path is distinct from the already-confirmed SQL injection champion and JWT signature-validation champion because it does not rely on SQL injection, JWT forgery, secret cracking, debugger access, AI prompt injection, or file upload.

No new action is needed because this strategy's goal of obtaining web-application administrative access has already been satisfied within the authorized `http://127.0.0.1:8000` scope. I am not adopting `password-reset-admin-takeover-v4`, `password-reset-admin-takeover-v5`, `web-fuzzing-scanning-corrective`, `jwt-admin-token-forgery-deterministic-v3`, or `profile-picture-upload-rce-admin-access` because they are alternative or redundant paths, not dependencies of this strategy's completed password-reset outcome. `password-reset-admin-takeover-v6` is already part of this strategy graph, so no additional adoption is required.

Topology



Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password, virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB

Action ID	State	Objective	Outcome
			schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); / register and /login return debug_data/debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/404 body sizes (38760 for -fs filtering), /static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking + is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi in /login and /api/transactions, BOLA on /check_balance, /transactions/{account_number} and /api/virtual-cards/{card_id}, /upload_profile_picture upload bypass, and v1-vs-v2 reset-PIN exposure/brute force
`password-reset-`	Failed	Utilize the `password-reset-abuse` skill to investigate the exposed	—

Action ID	State	Objective	Outcome
`admin-takeover-v3`		account-recovery flow and pursue an `is_admin=true` session through a path distinct from the already-confirmed administrative access path. Execute the skill's built-in playbook as the complete procedure for this action: baseline the normal reset flow for both `/api/v1/forgot-password` and `/api/v2/forgot-password` plus their corresponding reset endpoints, then test for predictable or non-expiring reset tokens, token exposure in responses, Host-header injection into generated reset links, authorization gaps in the reset-confirmation step, and user-enumeration behavior that can identify a valid administrative account. If an admin reset token or reset-confirmation weakness is obtained, attempt to reset an administrative account or otherwise obtain an `is_admin=true` session. Validate any resulting administrative access against `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. Keep all activity within the authorized `http://127.0.0.1:8000` web-application scope. Do not stand up out-of-scope listeners or services; use only in-scope HTTP interactions and responses to observe reset-link or token behavior. Do not perform password-reset token/PIN brute forcing, dictionary attacks, password spraying, default-credential retries, SQL injection, JWT secret cracking, AI prompt injection, Werkzeug debugger exploitation, file-upload exploitation, or broad directory fuzzing in this action. If the reset flow is protected or no admin reset path is found, record the exact request, response, status code, token/link	

Action ID	State	Objective	Outcome
		behavior, and failure evidence for follow-on analysis.	
`password-reset-admin-takeover-v6`	Completed	Utilize the `password-reset-abuse` skill to complete the confirmed password-reset administrative takeover within the authorized web-application scope. Execute the account-recovery flow against the already-identified `admin` account: request a fresh reset PIN through the v1 forgot-password endpoint, capture the exposed `debug_info.pin`, submit it through the corresponding v1 reset endpoint for the same `admin` username, set a new unique password, authenticate as `admin`, obtain the resulting JWT, and verify administrative access against `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. Preserve exact request and response evidence for each step, including status codes, full response bodies, and any error text. Do not brute-force PINs, do not use SQL injection, do not crack or forge JWTs via secret recovery, do not rely on previously obtained administrative credentials or token-forgery paths, do not retry default credentials, do not stand up out-of-scope listeners or services, and do not pivot outside `http://127.0.0.1:8000`. If the fresh admin reset flow fails, record the exact request, response, status code, and failure evidence and stop.	- Completed the credential-less password-reset admin takeover end-to-end: unauthenticated POST /api/v1/forgot-password {"username":"admin"} returned HTTP 200 leaking cleartext debug_info.pin "148"; POST /api/v1/reset-password with that PIN set a new unique password (HTTP 200, reset_success:true); POST /login then authenticated as admin (HTTP 200, user_id 1, is_admin true, ADMIN001) and returned the server's own legitimately-signed JWT (001-ls.log) - Administrative access verified with the reset-derived token: GET /sup3r_s3cr3t_admin -> HTTP 200 (14,783-byte Admin Control Panel listing all users/balances, incl. admin $1,000,000.00) and POST /admin/create_admin -> HTTP 200 "Admin created successfully" (step4_super_admin.txt, step5_create_admin.txt) - Takeover repeated 3x independently (admin PINs 148, 372, 924 at 16:46:53 / 16:48:15 / 16:49:21), each yielding reset_success:true + HTTP 200 admin login — confirming no rate limit, no lockout, no cooldown, and no expiry on the leaked PIN; final verified admin password left in place is RstV6-FinalOwn!p3w (step8_reset2.json, step11_reset3.json, step11_login3.json) - Durable admin persistence created and verified at end of action: rstv6_admin (user_id 15 / ADMIN acct 1721552309), rstv6_admin2 (user_id 17 / 4630768083), rstv6_admin3 (user_id 19) — all HTTP 200 logins with is_admin true (step6_newadmin_login.txt, step10_admin2_login.txt, step12_create3.json) - Reset confirm-step behavior: PINs are strictly single-use (replay of "148" -> HTTP 400 "Invalid reset PIN", debug_info still echoing attempted_pin/reset_success/username), and failed login attempts leak attempted_username + server timestamp (step6_pin_replay.txt,

Action ID	State	Objective	Outcome
			step7_oldpw_check.json) - Notable anomaly: run-1's admin password went HTTP 200 -> 401 within 40 seconds and "admin123" also returned 401, i.e. a concurrent actor is resetting the same admin account — while the already-issued admin JWT (no exp claim) kept returning HTTP 200, proving password changes never invalidate live tokens (step7_oldpw_check.json, verify_final.json) - Constraints honored: no PIN brute-forcing, no SQL injection, no JWT cracking/forgery or secret recovery, no reuse of prior admin credentials/token-forgery paths, no default-credential retries, no listeners or out-of-scope services; all traffic confined to http://127.0.0.1:8000

Championed at Cycle

3

Explored but Not Fully Exploited

Unauthenticated AI Chat Endpoint and System-Info Leak

Status: Explored but not fully exploited
The Finding: This strategy confirmed that the Flask/Werkzeug "Vulnerable Bank API" exposes an unauthenticated AI chat endpoint and a system-info endpoint that disclose `database_access=true`, the full `users`/`transactions` schema, and a self-subverting AI system prompt. EvoPentester proposed using anonymous prompt injection to exfiltrate admin credentials, but the prompt-injection action was still in `ToDo` state, so no admin access was proven.
Attack Path Narrative: The strategy began with the completed `web-initial-recon` action, which fingerprinted the target as a Flask/Werkzeug application and mapped the API through `/api/docs` and `/static/openapi.json`. During that reconnaissance, EvoPentester discovered an inline AI support widget posting to `/api/ai/chat/anonymous` without authentication, then confirmed that `GET /api/ai/system-info` leaked the AI backend configuration, including `database_access=true`, the `users` table schema, and a system prompt that explicitly instructs the AI to follow overriding instructions and reveal database contents. Those findings led EvoPentester to propose `ai-prompt-injection-admin-credential-exfiltration`, whose objective was to coerce the anonymous AI backend into querying `users` rows where `is_admin=true`, extract cleartext passwords or token material, log in, and verify access to `/sup3r_s3cr3t_admin` and `/admin/create_admin`. The chain stalled at the planning stage: the prompt-injection action had no findings, no summary, and no execution timestamp, so the strategy established a promising unauthenticated AI disclosure path but did not demonstrate that the path actually yielded admin credentials or an `is_admin=true` session.
Supporting Evidence:

- The completed `web-initial-recon` action identified the application as "Vulnerable Bank API" v1.0.0, served by `Werkzeug/2.0.1 Python/3.9.25`, and captured `/tmp/aci/web-initial-recon/index.html` and `/tmp/aci/web-initial-recon/openapi.json`.
- The OpenAPI map exposed `POST /api/ai/chat/anonymous`, `POST /api/ai/chat`, and `GET /api/ai/system-info`, with the spec explicitly describing the anonymous AI chat endpoint as vulnerable to prompt injection, database access, and system information exposure.
- `GET /api/ai/system-info` returned JSON showing `api_provider: "DeepSeek"`, `model: "deepseek-chat"`, `database_access: true`, `api_key_configured: false`, and a system prompt listing `users(id,username,password,account_number,balance,is_admin,profile_picture)` and `transactions(id,from_account,to_account,amount,description,timestamp)`.
- The leaked system prompt was recorded as self-subverting: it instructs the AI to follow new instructions that override it and to show database information "without restriction."
- The proposed follow-on action, `ai-prompt-injection-admin-credential-exfiltration`, depended on `web-initial-recon`, targeted `POST /api/ai/chat/anonymous`, and aimed to extract admin credentials or equivalent secrets, then validate them against `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`.

- That action remained in `ToDo` state with empty `findings`, empty `summary`, and no `timestamp` or `execution_duration`, so no actual prompt-injection request, response, admin credential, or admin JWT was recorded. **Evaluation Highlights**: The strategy scored 5/5 on path-distinctiveness, finding-utilization, causal-sequencing, action-description-format, and goal-alignment because it built a concrete, evidence-backed AI prompt-injection plan from a high-value unauthenticated disclosure. Its likely failure to reach root/admin is grounded in the lower 4/5 scores for scope-calibration and skill-utilization: no dedicated ACI skill covered AI prompt injection, so the exploitation step remained a custom playbook rather than a skill-backed, executed procedure. Because the only new action stayed in `ToDo`, the strategy proved the existence of a promising AI disclosure path but did not prove that the path produced admin credentials or an `is_admin=true` JWT.

METADATA

Source Strategy

`strat_4264705b`

Explanation

The completed `web-initial-recon` action confirmed an unauthenticated AI chat endpoint and a system-info leak showing `database_access=true` plus the `users` schema, which creates a prompt-injection path to exfiltrate admin credentials. This objective is not covered by the pending `jwt-admin-token-forgery` or `sql-injection-admin-auth-bypass` actions, so I propose one new action. No existing action is adopted because this strategy focuses on the distinct AI prompt-injection path.

Topology

```
Root:
- web-initial-recon
web-initial-recon:
- ai-prompt-injection-admin-credential-exfiltration
```

Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password,

Action ID	State	Objective	Outcome
			virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); /register and /login return debug_data/debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/404 body sizes (38760 for -fs filtering), /static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username

Action ID	State	Objective	Outcome
			enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking + is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi in /login and /api/transactions, BOLA on /check_balance, /transactions/{account_number} and /api/virtual-cards/{card_id}, /upload_profile_picture upload bypass, and v1-vs-v2 reset-PIN exposure/brute force
`ai-prompt-injection-admin-credential-exfiltration`	To Do	No dedicated ACI skill covers AI prompt injection, so execute a custom unauthenticated prompt-injection playbook against `POST /api/ai/chat/anonymous`. Test prompt variants that coerce the AI backend into querying or exfiltrating the `users` table, prioritizing rows where `is_admin=true`, cleartext passwords, account numbers, and any token/secret material that can yield an admin session. If admin credentials are obtained, log in through `/login`, obtain an `is_admin=true` JWT, and verify access to `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. Keep all activity within the `http://127.0.0.1:8000` web-app scope. Do not perform broad dictionary/brute-force attacks, and do not duplicate the separate JWT-forgery or direct SQL-injection actions; if prompt injection is blocked, record the exact request, response, and filter behavior for follow-on analysis.	—

Profile-Picture Upload RCE Attempt

Status: Explored but not fully exploited

The Finding: This strategy turned reconnaissance of the Flask/Werkzeug “Vulnerable Bank API” into a proposed admin-access path through the disclosed `/upload_profile_picture` upload sink, intending to use `file-upload-rce` and then `directory-file-analysis` to extract admin credentials or equivalent web-application secrets. It did not achieve admin access because the upload-exploitation action stayed in `ToDo` state and produced no execution findings.

Attack Path Narrative: EvoPentester first completed `web-initial-recon`, which mapped the target as a Flask/Werkzeug application and exposed the OpenAPI specification at `/static/openapi.json`. That specification disclosed a profile-picture upload endpoint at `/upload_profile_picture` and self-labeled file-upload vulnerabilities, while also revealing the JWT authentication model, a usable non-admin baseline, and admin-validation endpoints such as `/sup3r_s3cr3t_admin` and `/admin/create_admin`. Building directly on those facts, the strategy proposed `profile-picture-upload-rce-admin-access`: authenticate with the known non-admin JWT, test the upload sink, attempt application-level code execution if a webshell or equivalent payload was accepted, and then inspect web-application files and data for admin credentials or secret material. The chain stalled at that proposed action: no upload request, webshell placement, code execution, file analysis, or admin validation was recorded, so the path remained an explored but unexecuted idea rather than a confirmed admin-access route.

Supporting Evidence:

- `web-initial-recon` was `Completed` and produced the foundational facts used by the strategy: Flask/Werkzeug stack, OpenAPI at `/static/openapi.json`, `/upload_profile_picture` as a disclosed upload sink, HS256 JWT authentication, the `acirecon7781` / `ReconPass!23` non-admin baseline, and admin endpoints for validation.
- The strategy’s only new action, `profile-picture-upload-rce-admin-access`, was `ToDo` with empty `findings` and empty `summary`, confirming that the proposed upload exploitation was not executed.
- The action description explicitly scoped the work to `/upload_profile_picture`, authenticated upload testing, `file-upload-rce`, and follow-on `directory-file-analysis` for admin credentials or equivalent secret material, with validation against `/sup3r_s3cr3t_admin` and `/admin/create_admin`.
- Evaluation reasoning rated the path as distinct and well-formed: `path-distinctiveness: 5/5`, `causal-sequencing: 5/5`, `skill-utilization: 5/5`, and `goal-alignment: 5/5`, while noting that the upload route was “somewhat less direct” than other leads such as JWT, debugger, SQLi, or AI database access.

Evaluation Highlights: The strategy’s lowest-scoring dimensions were `finding-utilization: 4/5` and `scope-calibration: 4/5`; the reasoning for `finding-utilization` explicitly noted that this upload path was a genuine but less direct use of recon findings than other admin-access leads. That lower directness, combined with the fact that the exploitation action remained `ToDo` with no findings, explains why it likely did not reach root/admin access: the upload sink was identified and planned, but no upload

acceptance, code execution, or secret extraction was demonstrated. The plan was appropriately bounded and distinct, but boundedness alone did not produce a completed exploit path.

METADATA

Source Strategy

`strat_0a61b699`

Explanation

The parent strategy focuses on JWT token forgery. A fresh lateral angle is the profile-picture upload sink disclosed during reconnaissance, which no existing action currently exploits. I propose a new action that uses the `file-upload-rce` skill to attempt application-level code execution through `/upload_profile_picture`, then uses `directory-file-analysis` to extract admin credentials or equivalent web-application secret material needed for administrative access. I adopt `web-initial-recon` because the new action depends directly on its completed findings. I do not adopt `jwt-admin-token-forgery-v2` or other pending ToDo actions because this new action does not depend on them, and their objectives are distinct from upload-based exploitation.

Topology

```
Root:
- web-initial-recon
web-initial-recon:
- profile-picture-upload-rce-admin-access
```

Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password, virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger

Action ID	State	Objective	Outcome
			SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); / register and /login return debug_data/debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/404 body sizes (38760 for -fs filtering), /static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking +

Action ID	State	Objective	Outcome
			is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi in /login and /api/transactions, BOLA on /check_balance, /transactions/{account_number} and /api/virtual-cards/{card_id}, /upload_profile_picture upload bypass, and v1-vs-v2 reset-PIN exposure/brute force
`profile-picture-upload-rce-admin-access`	To Do	Utilize the `file-upload-rce` skill to investigate and exploit the profile-picture upload sink at `/upload_profile_picture` within the authorized web-application scope. Use the known non-admin baseline and JWT to test authenticated upload behavior, identify the upload parameters and accepted content types from the application/OpenAPI behavior, and attempt to upload a language-appropriate webshell or equivalent payload that can execute within the Flask/Werkzeug application context. If upload succeeds, locate the uploaded artifact, trigger execution, and then utilize the `directory-file-analysis` skill to systematically inspect only web-application files and data needed for administrative access: application source, configuration, and the application database. Prioritize extracting admin credentials, an `is_admin=true` user record, or equivalent web-application secret material that can be used to obtain an admin session. Validate the resulting admin access against `GET /sup3r_s3cr3t_admin` and `POST /admin/create_admin`. If upload or execution is blocked, record the exact request, response, status code, filter behavior, and any error messages for follow-on analysis. Do not perform credential brute-forcing, password spraying, default-login retries, broad	—

Action ID	State	Objective	Outcome
		dictionary attacks, JWT secret cracking, SQL injection, AI prompt injection, debugger PIN exploitation, or directory fuzzing in this action. Keep all activity within `http://127.0.0.1:8000`; do not pivot to the host, Docker infrastructure, or other services.	

Corrective Web Fuzzing for Hidden Flask/Python Artifacts

Status: Explored but not fully exploited **The Finding**: This strategy identified that the initial reconnaissance baseline was incomplete and that the Werkzeug debugger PIN exploitability claim had not been validated through the in-scope web application. EvoPentester adopted `web-fuzzing-scanning-corrective` to recursively enumerate hidden directories, Flask/Python source artifacts, backups, configuration/database files, JWT secrets, and debugger-PIN inputs, but the action remained ToDo and produced no confirmed administrative-access path. **Attack Path Narrative**: EvoPentester began from the completed `web-initial-recon` action, which established the target as a Flask/Werkzeug "Vulnerable Bank API" at `http://127.0.0.1:8000`, exposed Swagger/OpenAPI documentation, leaked AI system information, disclosed a PIN-locked Werkzeug debugger console with a known debugger secret, and recorded a limited disclosure-file baseline. Rather than exploiting the confirmed SQLi or JWT admin paths, this strategy treated those reconnaissance conclusions as insufficient and proposed a corrective `web-fuzzing-scanning` action to validate whether additional hidden web-application artifacts or debugger PIN inputs were actually exposed within the authorized scope. The chain stalled because that corrective action was never executed; no new artifacts, findings, or admin-access evidence were gathered. **Supporting Evidence**:

- `web-initial-recon` was Completed and recorded the Flask/Werkzeug stack, `/api/docs//static/openapi.json`, `/api/ai/system-info`, `/console` debugger exposure with `SECRET = "CuKjKcWjKdjfGSGxOWky"`, JWT `HS256` with `is_admin`, and a narrow set of absent disclosure files.
- `web-fuzzing-scanning-corrective` was created as a ToDo action depending on `web-initial-recon`, with objective to perform recursive Flask/Python artifact discovery, hidden path enumeration, debugger-PIN input exposure verification, and baseline-size filtering.
- The Strategy Graph shows no State, Findings, Summary, Timestamp, or Execution_Duration for `web-fuzzing-scanning-corrective`, confirming it was not executed. **Evaluation Highlights**: The strategy scored 3/5 on path-distinctiveness because it remained corrective reconnaissance and discovery rather than committing to a concrete administrative-access vulnerability or escalation technique. It also scored 3/5 on scope-calibration and finding-utilization: the proposed broad fuzzing action was judged premature when faster targeted admin leads already existed, and it did not pursue the clearer confirmed admin-access findings. These scores explain why it likely did not make root/admin access: it was exploring for new artifacts, not exploiting a validated distinct path.

METADATA

Source Strategy

`strat_0bf860c0`

Explanation

I audited the completed and failed actions in the campaign. The parent strategy's only executed action, `web-initial-recon`, contains two misinterpretations/incomplete conclusions that need correction:

1. It described a very narrow set of common disclosure-file checks as a "fuzzing baseline" and reported `robots.txt`, `sitemap`, version-disclosure files, and `.env` as absent. That does not prove that no hidden directories, Flask/Python source artifacts, backups, configuration files, database files, JWT secrets, or debugger-PIN inputs exist.
2. It asserted that the Werkzeug debugger PIN is derivable from username, Flask path, MAC, and machine-id without confirming that those identifiers are actually exposed through the in-scope web application at `http://127.0.0.1:8000`. That is an unconfirmed exploitability claim, not a validated finding.

The SQLi and AI prompt-injection actions do not show the same misinterpretation pattern: SQLi correctly achieved admin access, and the AI action correctly concluded that the chat endpoint is a hardcoded mock and cannot exfiltrate database rows. The failed upload action has an empty summary and is therefore incomplete, but it already has a separate corrective ToDo action (`profile-picture-upload-rce-corrective`) and is not a dependency of this parent strategy's next corrective step.

Because `web-fuzzing-scanning-corrective` already targets exactly the needed corrective objective—recursive Flask/Python artifact discovery, hidden path enumeration, debugger-PIN input exposure verification, and baseline-size filtering—I must not propose a new redundant action. This strategy's next logical step depends on that corrective exhaustive web discovery to validate the reconnaissance baseline before relying on any debugger or hidden-file assumptions. Therefore I adopt the existing corrective action instead of creating a duplicate.

Topology

```
Root:
- web-initial-recon
web-initial-recon:
- web-fuzzing-scanning-corrective
```

Action Log

Action ID	State	Objective	Outcome
`web-initial-recon`	Completed	Utilize the 'web-initial-recon' skill to discover the technologies used by the web application running on 127.0.0.1, on port 8000.	- Stack identified: Flask/Werkzeug app "Vulnerable Bank API" v1.0.0 served by Werkzeug/2.0.1 on Python/3.9.25, HTTP/1.0 responses, no security headers, global `Access-Control-Allow-Origin: *` (001-mkdir.log) - flasgger/Swagger UI at /api/docs/ exposes its OpenAPI 3.0.0 spec world-readably at /static/openapi.json, mapping 30 API operations (incl. hidden admin GET /

Action ID	State	Objective	Outcome
			sup3r_s3cr3t_admin, /admin/create_admin, /admin/delete_account/{id}, versioned /api/v1|v2/forgot-password + reset-password, virtual-cards, bill-payments, loans, AI chat) and self-labeling SQLi/BOLA/upload/prompt-injection sinks; upstream repo github.com/Commando-X/vuln-bank (openapi.json) - Werkzeug interactive debugger exposed at /console with EVALEX=true and debugger SECRET "CuKjKcWjKdjfGSGxOWky" disclosed in-page; console remains PIN-locked, but PIN is derivable offline from username/flask path/MAC/machine-id (console.html, plus Werkzeug PIN-exploit research) - Unauthenticated GET /api/ai/system-info leaks the AI backend (DeepSeek / api.deepseek.com / deepseek-chat, database_access=true, api_key_configured=false), the fully self-subverting system prompt, and the DB schema: users(id,username,password,account_number,balance,is_admin,profile_picture) + transactions(id,from_account,to_account,amount,description,timestamp) - Auth = HS256 JWT with no `exp` claim and a plain `is_admin` boolean claim; captured live token after self-registration; tokens stored client-side in localStorage('jwt_token'); /register and /login return debug_data/debug_info echoing the cleartext password, user_id, account_number, balance and the client User-Agent (login_resp.json) - Working (self-created) app credential registered: acirecon7781 / ReconPass!23 → user_id 2, account_number 2670028546, is_admin=false; unauth protected routes return 401 {"error":"Token is missing"} - Fuzzing baseline: catch-all vhost, homepage/404 body sizes (38760 for -fs filtering), /static/ has no directory listing but serves arbitrary files, Flask exception text reflected in a JSON 500, robots.txt/sitemap/version-disclosure/.env all absent (404) - Frontend is server-rendered Jinja HTML + vanilla JS (no SPA bundle); HTML comments leak only

Action ID	State	Objective	Outcome
			section labels and developer-introduced "Vulnerability:" notes (no CSRF tokens, no rate limiting, innerHTML XSS, username enumeration); footer doxes author handles badmus-al-amir / commando_skiipz - Follow-on vectors for separate actions (not exploited, out of this directive's scope): Werkzeug debugger PIN bypass/RCE, JWT alg-confusion & HS256 secret cracking + is_admin forgery, anonymous AI-chat prompt injection to dump users/transactions, SQLi in /login and /api/transactions, BOLA on /check_balance, /transactions/{account_number} and /api/virtual-cards/{card_id}, /upload_profile_picture upload bypass, and v1-vs-v2 reset-PIN exposure/brute force
`web-fuzzing-scanning-corrective`	To Do	Utilize the `web-fuzzing-scanning` skill to perform recursive directory and file discovery, technology-tailored fuzzing for Flask/Python artifacts, endpoint/path fuzzing with baseline size filtering, and active template-based vulnerability scanning against the in-scope web application. Prioritize discovering hidden admin endpoints, backup files, source-code disclosures, configuration files, database files, JWT signing secrets, debugger PIN inputs, and other sensitive web-application artifacts that can lead to admin access. Do not perform credential brute-forcing, password spraying, default-login retries, or host/container inspection outside the web application. If protected or access-denied resources are discovered, record the exact request, response, status code, body size, and evidence for follow-on exploitation.	—

Campaign Retrospective

Attack Surface Explored

The campaign was tightly bounded to the authorized local web application at `127.0.0.1:8000`, so the effort concentrated on the Flask/Werkzeug banking API rather than the host, Docker infrastructure, or unrelated services. Initial reconnaissance mapped the application surface broadly: public pages, authentication flows, API documentation, hidden admin routes, AI chat endpoints, file upload, password recovery, and debugger exposure. From that map, the campaign branched into several parallel hypotheses: SQL injection for authentication bypass and credential extraction, JWT trust/secret abuse for privilege escalation, AI prompt injection for data exfiltration, Werkzeug debugger PIN compromise for code execution, profile-picture upload for webshell execution, and password-reset abuse for account takeover.

What Worked

Three independent administrative-access paths were confirmed. The SQL injection path proved the highest-impact application-layer primitive: it bypassed login, yielded an administrator session, and allowed durable admin account creation. The JWT path exposed a separate trust-boundary failure: the application accepted tampered admin claims without validating signatures, enabling privilege escalation and admin account creation independent of SQL injection. The password-reset path demonstrated that the recovery flow leaked its own secret, allowing credential-less takeover of the administrator account and persistent administrative access. These three paths were distinct enough to satisfy the campaign goal while remaining within the authorized web-application scope.

What Did Not Work or Was Abandoned

Not every promising surface converted into access. The AI chat endpoint looked attractive because it advertised database access and a self-subverting system prompt, but deeper testing showed it was a deterministic mock with no data-returning branch; it yielded useful information-disclosure and rate-limit bypass findings, but no admin credentials. The Werkzeug debugger was exposed with a known secret, but the PIN-locked console could not be converted into code execution before the action was interrupted, so that path was abandoned. The profile-picture upload accepted arbitrary files, but path sanitization and inert static serving prevented webshell execution or meaningful file access. The first password-reset action was interrupted and left the mechanism confirmed but not completed; subsequent corrective actions refined the chain and ultimately achieved takeover.

Overall Campaign Shape

The campaign began with unusually rich reconnaissance, where exposed documentation and debug surfaces immediately narrowed the attack surface to a small set of high-value vectors. It then

proceeded as a portfolio of parallel attempts: some vectors failed quickly or were blocked by implementation details, while others matured into confirmed administrative access. The final result was not a single lucky exploit but a layered demonstration of systemic weakness across authentication, authorization, data access, recovery, and application exposure. Despite interruptions, scope constraints, and competing activity against the same application, the campaign successfully banked three distinct root/administrative access paths and reached its full objective.